

Arsitektur & Spesifikasi Aplikasi Akuntansi Serupa Accurate Online

https://gemini.google.com/app/636153cc6131a6d3

User prompt: riset mengenai accurate online. saya ingin membuat aplikasi serupa dengan accurate online. laporkan secara detail, setiap akun, fitur, database, flow, dan setiap detailnya. dengan tujuan, saya bisa minta AI lain untuk membuat yang sama dengan accurate online, namun di modifikasi lebih lengkap lagi.

Response: Berikut adalah laporan riset dan spesifikasi sistem (berbasis arsitektur **Accurate Online**) yang disusun secara terstruktur. Dokumen ini dirancang sebagai *System Requirement Specification* (SRS) sehingga Anda dapat langsung memberikannya sebagai *prompt* atau panduan kepada AI pengembang (AI Coder) untuk membangun aplikasinya.

1. Arsitektur Database & Struktur Entitas Inti

Untuk mereplikasi sistem akuntansi ERP, sistem harus menggunakan arsitektur *Double-Entry Accounting* di mana setiap transaksi selalu seimbang (*balanced*). Berikut adalah tabel-tabel utama yang wajib dibuat:

A. Master Data (Data Referensi)

- **users & roles:** Manajemen hak akses *multi-user* (misal: kasir tidak bisa melihat laba rugi, manajer memiliki akses *Approval Workflow*).
- **branches & warehouses:** Mendukung multi-cabang dan multi-gudang.
- **chart_of_accounts (COA):** Tabel hierarki akun perkiraan (Induk & Anak).
- **contacts:** Entitas gabungan untuk Pelanggan (*Customers*), Pemasok (*Vendors*), dan Karyawan (*Employees*).
- **items & item_categories:** Master data barang dan jasa, mendukung satuan bertingkat (Pcs > Box > Karton).

B. Tabel Transaksional (Alur Dokumen)

- **Penjualan (Order-to-Cash):** sales_quotations → sales_orders → delivery_orders → sales_invoices → customer_receipts.
- **Pembelian (Procure-to-Pay):** purchase_requisitions → purchase_orders → item_receipts → purchase_invoices → vendor_payments.

C. Tabel Jurnal & Buku Besar (Core Accounting Engine)

Semua tabel transaksional di atas tidak boleh menghitung saldo secara mandiri, melainkan harus bermuara pada *General Ledger* (Buku Besar):

- **journal_entries:** Header (Nomor Jurnal, Tanggal, Referensi Dokumen, Keterangan).
- **journal_entry_lines:** Detail mutasi (Akun ID, Debit, Kredit). *Rule wajib: Total Debit harus sama dengan Total Kredit.*

2. Rincian Modul Utama & Fitur Spesifik

Untuk membuat sistem ini lengkap, AI Anda perlu membangun modul-modul berikut beserta logika internalnya:

Modul	Fitur Wajib yang Harus Direplikasi
Buku Besar	Pembuatan Jurnal Umum, Histori Akun, Log Aktivitas Transaksi, dan Rekam Pencatatan Gaji (termasuk <i>tax/PPh21</i>).

Modul	Fitur Wajib yang Harus Direplikasi
Kas & Bank	Pencatatan penerimaan/pembayaran lain, transfer antar bank, dan Rekonsiliasi Bank Otomatis (mencocokkan rekening koran dengan pencatatan sistem).
Penjualan	Multi-diskon (misal: 10% + 5%), perhitungan PPN otomatis, tukar tambah (<i>trade-in</i>), target penjualan, dan perhitungan komisi tenaga penjual.
Pembelian	<i>Aging Account Payable</i> (umur utang pemasok), limit batas utang, retur pembelian, dan pembayaran parsial.
Persediaan	Perhitungan Harga Pokok Penjualan (HPP) otomatis dengan metode FIFO atau Moving Average , Penyesuaian Stok (<i>Stock Opname</i>), dan pergerakan antar gudang.
Aset Tetap	Perhitungan penyusutan/depresiasi otomatis setiap akhir bulan berdasarkan tipe aset (Garis Lurus / Saldo Menurun).
Laporan	Pembuat laporan dinamis untuk: Neraca, Laba/Rugi, Arus Kas, Kartu Stok, dan Buku Besar Pembantu.

Catatan Teknis untuk AI Pengembang: Pastikan arsitektur API dibangun secara modular (*Microservices* atau *Modular Monolith*) agar ketika ada kesalahan di modul Persediaan, tidak merusak fungsi Buku Besar.

3. Alur Logika Sistem (Business Workflows)

Saat memerintahkan AI, jelaskan bahwa perpindahan dokumen akan otomatis memicu perpindahan jurnal. Berikut adalah dua *workflow* paling krusial:

- **Siklus Pembelian (Faktur Pemasok):** Saat *Purchase Invoice* dibuat, sistem tidak hanya mencatat dokumen tagihan, tetapi *engine* harus otomatis membuat jurnal:
 - (Debit) Persediaan Barang
 - (Debit) PPN Masukan
 - (Kredit) Utang Usaha
- **Siklus Penjualan (Pengiriman Barang & Faktur):** Saat barang dikirim (*Delivery Order*) dan ditagihkan (*Sales Invoice*), sistem harus melakukan 4 mutasi sekaligus:
 - (Debit) Piutang Usaha
 - (Kredit) Penjualan
 - (Debit) Harga Pokok Penjualan (HPP) → *Nilai diambil berdasarkan FIFO/Average*.
 - (Kredit) Persediaan Barang

4. Standar Chart of Accounts (COA)

AI harus menanamkan *default template* struktur akun 5 digit agar pengguna baru tidak perlu membangun dari nol. Format standar klasifikasinya adalah:

- **1xxxx - Aset** (11xxx: Aset Lancar spt Kas/Bank, 12xxx: Piutang, 13xxx: Persediaan, 15xxx: Aset Tetap)
- **2xxxx - Kewajiban/Liabilitas** (Utang Pemasok, Utang Pajak)
- **3xxxx - Ekuitas** (Modal, Laba Ditahan)
- **4xxxx - Pendapatan** (Penjualan Produk/Jasa)
- **5xxxx - Harga Pokok Penjualan (HPP)**
- **6xxxx - Beban Operasional** (Gaji, Listrik, Sewa)

- 7xxxx - Pendapatan & Beban Lain-lain
- 8xxxx - Pajak Penghasilan Badan

5. Rekomendasi Modifikasi (Lebih Baik dari Accurate)

Kelemahan sistem ERP tradisional adalah entri data manual yang memakan waktu. Anda bisa menginstruksikan AI untuk menambahkan fitur-fitur "Next-Gen" berikut agar aplikasi Anda lebih superior:

1. **AI-Powered OCR (Optical Character Recognition):** Pengguna cukup memfoto struk atau PDF tagihan pemasok (*invoice*), lalu sistem otomatis membacanya dan membuat *draft* jurnal tanpa pengetikan manual.
2. **Integrasi Manajemen SDM (HRIS) yang Kuat:** Accurate fokus pada pencatatan angka gaji. Aplikasi Anda bisa ditambahkan fitur absensi berbasis lokasi (GPS) dan cuti karyawan yang langsung terhubung otomatis ke jurnal penggajian di akhir bulan.
3. **Predictive Analytics (Peramalan):** Tambahkan dasbor yang menggunakan algoritma untuk memprediksi kapan kas perusahaan akan habis (*Cash Flow Forecasting*) atau peringatan otomatis jika bahan baku hampir habis sebelum masa tenggang (*Reorder Point* cerdas).
4. **Webhooks & API Builder Visual:** Berikan fasilitas bagi pengguna (tanpa *coding*) untuk menghubungkan sistem ERP Anda ke aplikasi lain seperti WhatsApp (untuk notifikasi tagihan ke pelanggan) atau platform *e-commerce*.

Sources

1. [Memahami Fitur Accurate Online untuk Efisiensi Keuangan Anda](#)
2. [Fitur & Modul Yang Terdapat di Accurate Accounting Software](#)
3. [Modul dan Fitur Accurate Online | PDF - Scribd](#)

User prompt: Buat rancangan kode skema database SQL (DDL) lengkap beserta relasinya (Foreign Keys) untuk modul Buku Besar, Penjualan, dan Persediaan.

Response: Berikut adalah rancangan skema database SQL (Data Definition Language / DDL) untuk inti dari aplikasi ERP Anda. Skema ini dirancang menggunakan standar SQL relasional (kompatibel dengan PostgreSQL atau MySQL) dengan tipe data presisi tinggi (`DECIMAL(15,2)`) yang wajib digunakan untuk nilai uang agar tidak terjadi kebocoran pembulatan.

Saya membaginya menjadi tiga area utama yang saling terhubung: **Master & Persediaan, Penjualan, dan Buku Besar.**

```
-- =====
-- 1. MODUL BUKU BESAR (GENERAL LEDGER) & MASTER DATA
-- =====

-- Tabel Induk Akun (Chart of Accounts)
CREATE TABLE chart_of_accounts (
    id INT PRIMARY KEY AUTO_INCREMENT,
    account_code VARCHAR(20) UNIQUE NOT NULL,
    account_name VARCHAR(150) NOT NULL,
    account_type ENUM('ASSET', 'LIABILITY', 'EQUITY', 'REVENUE', 'COGS', 'EXPENSE') NOT NULL,
    parent_account_id INT NULL, -- Untuk hierarki akun (Sub-akun)
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (parent_account_id) REFERENCES chart_of_accounts(id)
);

-- Header Jurnal Umum
CREATE TABLE journal_entries (
    id INT PRIMARY KEY AUTO_INCREMENT,
    journal_number VARCHAR(50) UNIQUE NOT NULL,
    transaction_date DATE NOT NULL,
    reference_type VARCHAR(50), -- Cth: 'SALES_INVOICE', 'MANUAL_JOURNAL'
    reference_id INT,           -- ID dari tabel referensi (fleksibel)
```

```

        description TEXT,
        status ENUM('DRAFT', 'POSTED', 'VOID') DEFAULT 'POSTED',
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

-- Detail/Baris Jurnal Umum (Harus Balance: Total Debit = Total Kredit)
CREATE TABLE journal_entry_lines (
    id INT PRIMARY KEY AUTO_INCREMENT,
    journal_entry_id INT NOT NULL,
    account_id INT NOT NULL,
    debit DECIMAL(15, 2) DEFAULT 0.00,
    credit DECIMAL(15, 2) DEFAULT 0.00,
    memo VARCHAR(255),
    FOREIGN KEY (journal_entry_id) REFERENCES journal_entries(id) ON DELETE CASCADE,
    FOREIGN KEY (account_id) REFERENCES chart_of_accounts(id)
);

-- =====
-- 2. MODUL PERSEDIAAN (INVENTORY)
-- =====

-- Master Gudang
CREATE TABLE warehouses (
    id INT PRIMARY KEY AUTO_INCREMENT,
    warehouse_code VARCHAR(20) UNIQUE NOT NULL,
    warehouse_name VARCHAR(100) NOT NULL,
    address TEXT
);

-- Master Barang & Jasa
CREATE TABLE items (
    id INT PRIMARY KEY AUTO_INCREMENT,
    item_code VARCHAR(50) UNIQUE NOT NULL,
    item_name VARCHAR(200) NOT NULL,
    item_type ENUM('INVENTORY', 'NON_INVENTORY', 'SERVICE') NOT NULL,
    base_unit VARCHAR(20) NOT NULL, -- Cth: 'PCS', 'BOX'
    -- Default Akun Jurnal (Auto-Posting)
    inventory_account_id INT NULL, -- Akun Persediaan
    sales_account_id INT NOT NULL, -- Akun Pendapatan
    cogs_account_id INT NULL, -- Akun HPP
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (inventory_account_id) REFERENCES chart_of_accounts(id),
    FOREIGN KEY (sales_account_id) REFERENCES chart_of_accounts(id),
    FOREIGN KEY (cogs_account_id) REFERENCES chart_of_accounts(id)
);

-- Kartu Stok / Pergerakan Barang (Inventory Ledger)
CREATE TABLE inventory_transactions (
    id INT PRIMARY KEY AUTO_INCREMENT,
    item_id INT NOT NULL,
    warehouse_id INT NOT NULL,
    transaction_date DATE NOT NULL,
    reference_type VARCHAR(50), -- Cth: 'SALES_INVOICE', 'RECEIVE_ITEM'
    reference_id INT,
    qty_in DECIMAL(10, 2) DEFAULT 0,
    qty_out DECIMAL(10, 2) DEFAULT 0,
    cogs_unit_price DECIMAL(15, 2) NOT NULL, -- Harga Pokok saat transaksi
    FOREIGN KEY (item_id) REFERENCES items(id),
    FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);

-- =====
-- 3. MODUL PENJUALAN (SALES)
-- =====

```

```
-- =====

-- Master Pelanggan
CREATE TABLE customers (
  id INT PRIMARY KEY AUTO_INCREMENT,
  customer_code VARCHAR(50) UNIQUE NOT NULL,
  customer_name VARCHAR(150) NOT NULL,
  email VARCHAR(100),
  receivable_account_id INT NOT NULL, -- Akun Piutang Default
  FOREIGN KEY (receivable_account_id) REFERENCES chart_of_accounts(id)
);

-- Header Faktur Penjualan (Sales Invoice)
CREATE TABLE sales_invoices (
  id INT PRIMARY KEY AUTO_INCREMENT,
  invoice_number VARCHAR(50) UNIQUE NOT NULL,
  transaction_date DATE NOT NULL,
  due_date DATE NOT NULL,
  customer_id INT NOT NULL,
  subtotal DECIMAL(15, 2) NOT NULL,
  tax_amount DECIMAL(15, 2) DEFAULT 0.00,
  total_amount DECIMAL(15, 2) NOT NULL,
  status ENUM('UNPAID', 'PARTIAL', 'PAID', 'VOID') DEFAULT 'UNPAID',
  journal_entry_id INT NULL, -- Terhubung langsung ke Bukti Jurnal
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (customer_id) REFERENCES customers(id),
  FOREIGN KEY (journal_entry_id) REFERENCES journal_entries(id)
);

-- Detail Faktur Penjualan (Sales Invoice Lines)
CREATE TABLE sales_invoice_lines (
  id INT PRIMARY KEY AUTO_INCREMENT,
  sales_invoice_id INT NOT NULL,
  item_id INT NOT NULL,
  warehouse_id INT NOT NULL,
  quantity DECIMAL(10, 2) NOT NULL,
  unit_price DECIMAL(15, 2) NOT NULL,
  discount_amount DECIMAL(15, 2) DEFAULT 0.00,
  line_total DECIMAL(15, 2) NOT NULL, -- (qty * unit_price) - discount
  FOREIGN KEY (sales_invoice_id) REFERENCES sales_invoices(id) ON DELETE CASCADE,
  FOREIGN KEY (item_id) REFERENCES items(id),
  FOREIGN KEY (warehouse_id) REFERENCES warehouses(id)
);
```

Penjelasan Logika Relasi (Penting Disampaikan ke AI Pengembang)

- 1. Auto-Posting Penjualan ke Jurnal:** Perhatikan kolom `journal_entry_id` pada tabel `sales_invoices`. Saat sebuah Faktur Penjualan di-*posting* ke *database*, *backend* harus memicu fungsi yang secara otomatis menyisipkan data ke `journal_entries` dan `journal_entry_lines`. Jika jurnal gagal dibuat, maka *invoice* juga harus dibatalkan (*Database Transaction Rollback*).
- 2. Pemetaan Akun Otomatis:** Tabel `items` memiliki `inventory_account_id`, `sales_account_id`, dan `cogs_account_id`. Ini memastikan saat barang A dijual, sistem tahu persis akun pendapatan mana yang di-Kredit dan akun HPP mana yang di-Debit tanpa perlu intervensi pengguna.
- 3. Pemisahan Stok dan Keuangan:** Tabel `inventory_transactions` mencatat masuk-keluarnya barang secara fisik beserta nilai HPP pada saat transaksi (sangat penting untuk algoritma FIFO/Average). Sistem tidak menghitung sisa stok dari tabel penjualan, melainkan dari $SUM(qty_in) - SUM(qty_out)$ pada tabel transaksi *inventory* ini.